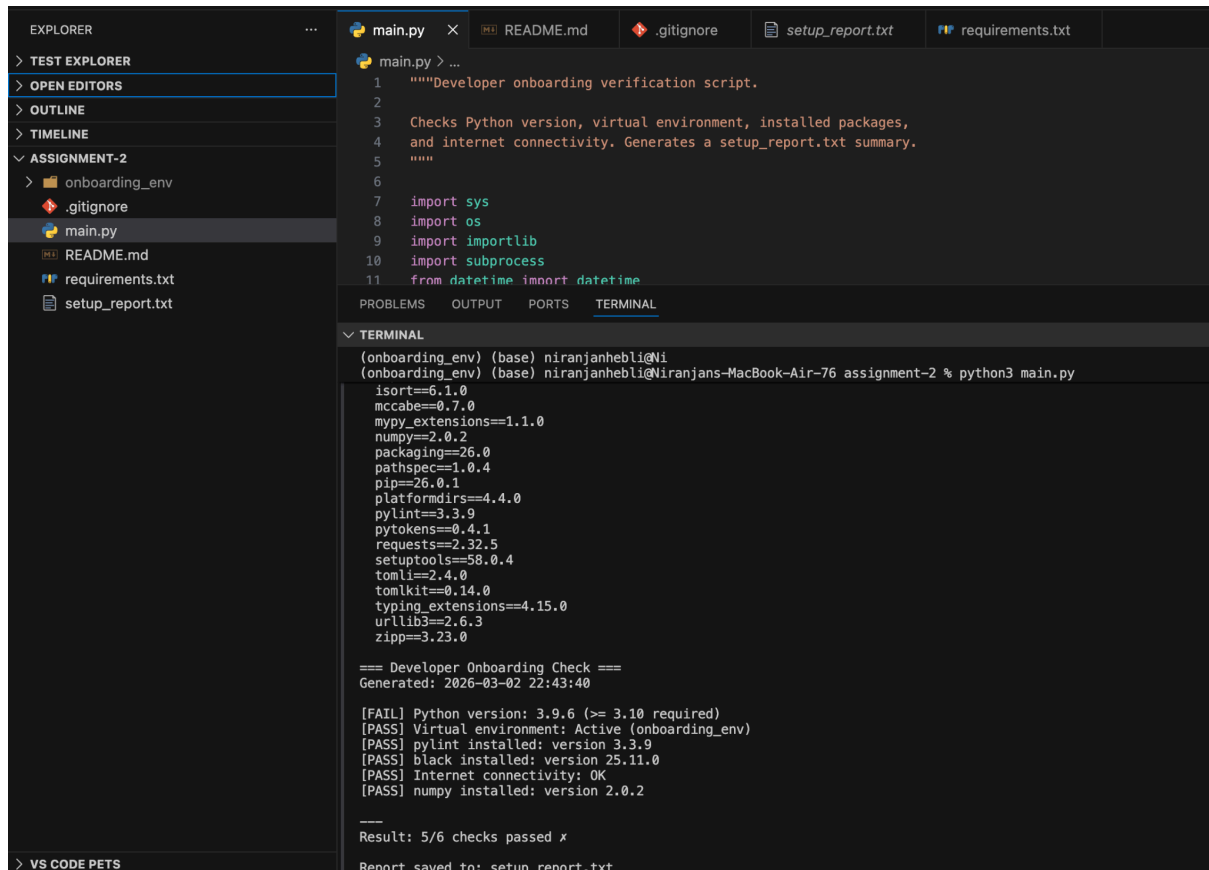


# Python Dev Environment Setup

## Part A: Concept Application

Github Link:- [https://github.com/NiranjanHebli/DAY7\\_AM\\_Assignment](https://github.com/NiranjanHebli/DAY7_AM_Assignment)

Output Screenshot:-



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'onboarding\_env' with files: .gitignore, main.py, README.md, requirements.txt, and setup\_report.txt. The main.py file is open in the editor, showing a Python script for developer onboarding verification. The terminal output shows the execution of the script, which checks various system and environment details and generates a setup\_report.txt file.

```
main.py > ...
1 """Developer onboarding verification script.
2
3 Checks Python version, virtual environment, installed packages,
4 and internet connectivity. Generates a setup_report.txt summary.
5 """
6
7 import sys
8 import os
9 import importlib
10 import subprocess
11 from datetime import datetime

PROBLEMS OUTPUT PORTS TERMINAL

▼ TERMINAL
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 % python3 main.py

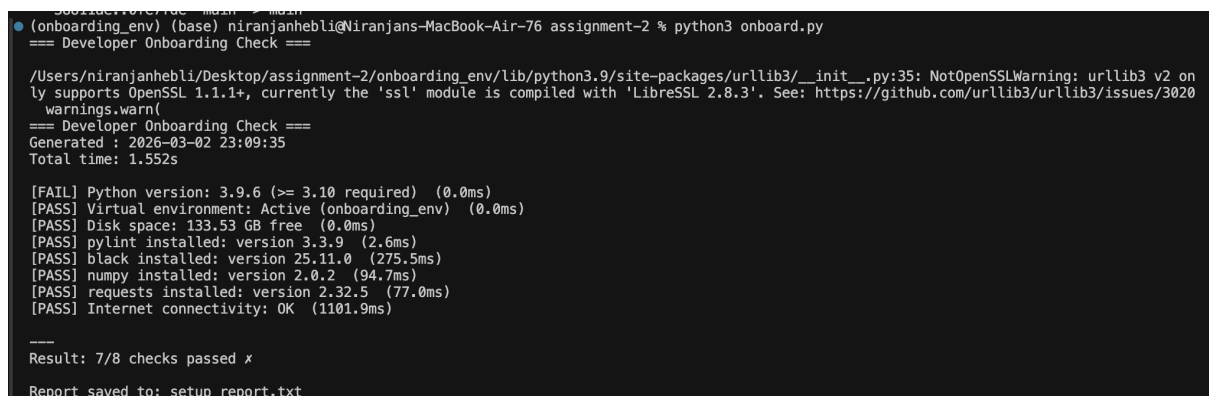
isort==6.1.0
mccabe==0.7.0
mypy_extensions==1.1.0
numpy==2.0.2
packaging==26.0
pathspec==1.0.4
pip==26.0.1
platformdirs==4.4.0
pylint==3.3.9
pytokens==0.4.1
requests==2.32.5
setuptools==58.0.4
tomli==2.4.0
tomlkit==0.14.0
typing_extensions==4.15.0
urllib3==2.6.3
zipp==3.23.0

=== Developer Onboarding Check ===
Generated: 2026-03-02 22:43:40

[FAIL] Python version: 3.9.6 (>= 3.10 required)
[PASS] Virtual environment: Active (onboarding_env)
[PASS] pylint installed: version 3.3.9
[PASS] black installed: version 25.11.0
[PASS] Internet connectivity: OK
[PASS] numpy installed: version 2.0.2

---
Result: 5/6 checks passed x
Report saved to: setup_report.txt
```

## Part B: Stretch Problem



The screenshot shows a terminal window with the output of a Python script. The script performs a developer onboarding check, including checking the Python version, virtual environment, installed packages, and internet connectivity. The output shows that 7 out of 8 checks passed, and a setup\_report.txt file was generated.

```
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 % python3 onboard.py
=== Developer Onboarding Check ===

/Users/niranjanhebli/Desktop/assignment-2/onboarding_env/lib/python3.9/site-packages/urllib3/_init_.py:35: NotOpenSSLWarning: urllib3 v2 on
ly supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
=== Developer Onboarding Check ===
Generated: 2026-03-02 23:09:35
Total time: 1.552s

[FAIL] Python version: 3.9.6 (>= 3.10 required) (0.0ms)
[PASS] Virtual environment: Active (onboarding_env) (0.0ms)
[PASS] Disk space: 133.53 GB free (0.0ms)
[PASS] pylint installed: version 3.3.9 (2.6ms)
[PASS] black installed: version 25.11.0 (275.5ms)
[PASS] numpy installed: version 2.0.2 (94.7ms)
[PASS] requests installed: version 2.32.5 (77.0ms)
[PASS] Internet connectivity: OK (1101.9ms)

---
Result: 7/8 checks passed x
Report saved to: setup_report.txt
```

## Part C: Interview Ready

### Q1: Conceptual — Virtual Environments

A Python virtual environment is like a **sandbox** where developers can install packages without affecting the rest of the system. Developers use them to avoid conflicts between projects, since different projects may need different versions of the same library. Without one, installing packages globally can cause version clashes and break other projects.

### Q2: Coding — Fix and improve this code

---

```
# This code has 5 issues. Find and fix ALL of them.  
# Then make it pass Pylint with score >= 8/10.
```

```
import os, sys, json  
  
def checkVersion():  
    v = sys.version_info  
    if v.major >= 3 and v.minor>=11:  
        return "Good"  
    else:  
        return "Bad"  
  
result = checkVersion()  
temp = 42  
print( "Version status:" , result )
```

List the 5 issues and provide the corrected code.

#### The 5 Issues Identified

1. Multiple imports on one line (import os, sys, json) → violates PEP8 style.
2. Unused imports (os, json) → should be removed.
3. Function name not in snake\_case (checkVersion) → should be check\_version.
4. Unused variable (temp = 42) → unnecessary, lowers lint score.
5. Docstrings missing → Pylint will complain about missing function/module docstrings.

```
script.py > ...
1  # This code has 5 issues. Find and fix ALL of them.
2  # Then make it pass Pylint with score >= 8/10.
3  import sys
4
5
Windsurf: Refactor | Explain | X
6  def check_version():
7      """Check if Python version is >= 3.11."""
8      v = sys.version_info
9      if v.major >= 3 and v.minor >= 11:
10         return "Good"
11     return "Bad"
12
13
Windsurf: Refactor | Explain | X
14 def main():
15     """Main entry point."""
16     result = check_version()
17     print("Version status:", result)
18
19
20 if __name__ == "__main__":
21     main()
22
```

PROBLEMS OUTPUT PORTS TERMINAL

TERMINAL zsh + v

```
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 % pylint script.py
***** Module script
script.py:1:0: C0114: Missing module docstring (missing-module-docstring)

-----
Your code has been rated at 9.09/10 (previous run: 9.09/10, +0.00)

(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 %
```

### Q3: Debug/Analyze

A junior developer says: "I installed numpy but Python says ModuleNotFoundError: No module named 'numpy'". Give 3 possible causes and the diagnostic command for each.

Ans:-

1. Using the Wrong Version of Python

Most computers have multiple versions of Python installed. He might have installed NumPy for Python 3.10, but is trying to run his script using Python 3.12. Since each version has its own folder for "extras," the new version can't see what he has installed for the old one.

- Diagnostic Command: `python --version` (Compare this to the version shown when run `pip --version`)

## 2. Virtual Environment Not Activated

Developers often use "Virtual Environments" to keep projects separate. If he has installed NumPy while a virtual environment was active, but he is now trying to run the script in a global terminal (or vice versa), Python won't be able to find the module.

- Diagnostic Command: `pip list` (This shows every library the current environment can see.)

## 3. Pip and Python Are Not Linked

Sometimes the command `pip` points to a completely different location than the command `python`. This happens often on macOS and Linux. `pip` is probably sending it to a hidden system folder that his script doesn't check.

- Diagnostic Command: `which python` and `which pip` (On Windows, use `where python` and `where pip`). If the file paths lead to different folders, they are not linked.

# Part D: AI-Augmented Task

## Prompt Used:-

Act as an expert Python developer. Generate a `.pylintrc` configuration file specifically for a beginner-level AI/Machine Learning project.

Please follow these requirements:

- \* Relaxed Style: Increase the `max-line-length` to 120 characters to accommodate long data processing strings and deep learning layers.
- \* Ignore Specific Warnings: Disable C0114, C0115, and C0116 (missing docstrings) so the beginner isn't forced to document every single function immediately.
- \* Naming Conventions: Adjust naming regex to be more lenient with variable names (e.g., allow short names like `i`, `j`, `X`, `y`, which are standard in ML).
- \* Similarity Threshold: Increase the `min-similarity-lines` to 10 to avoid flagging repetitive (but necessary) boilerplate code often found in model training loops.
- \* Plugin Support: Include `pylint.extensions.docparams` and ensure it plays well with common ML libraries like NumPy, Pandas, and PyTorch/TensorFlow. stay within the 100-line constraint while maintaining essential configuration guidance.

## Github

Link:-[https://github.com/NiranjanHebli/DAY7\\_AM\\_Assignme](https://github.com/NiranjanHebli/DAY7_AM_Assignme)  
[nt/blob/main/.pylintrc](https://github.com/NiranjanHebli/DAY7_AM_Assignme/blob/main/.pylintrc)

```
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 % pylint onboard.py
```

### Raw metrics

| type      | number | %     | previous | difference |
|-----------|--------|-------|----------|------------|
| code      | 120    | 65.22 | NC       | NC         |
| docstring | 23     | 12.50 | NC       | NC         |
| comment   | 8      | 4.35  | NC       | NC         |
| empty     | 33     | 17.93 | NC       | NC         |

### Duplication

|                          | now   | previous | difference |
|--------------------------|-------|----------|------------|
| nb duplicated lines      | 0     | NC       | NC         |
| percent duplicated lines | 0.000 | NC       | NC         |

### Messages by category

| type       | number | previous | difference |
|------------|--------|----------|------------|
| convention | 0      | NC       | NC         |
| refactor   | 0      | NC       | NC         |
| warning    | 1      | NC       | NC         |
| error      | 0      | NC       | NC         |

### % errors / warnings by module

| module  | error | warning | refactor | convention |
|---------|-------|---------|----------|------------|
| onboard | 0.00  | 100.00  | 0.00     | 0.00       |

### Messages

| message id                     | occurrences |
|--------------------------------|-------------|
| f-string-without-interpolation | 1           |

Your code has been rated at 9.90/10

```
221 ignore-imports=yes
222
223
224 [DESIGN]
225
226 # =====
227 # complexity limits - generous enough for ML, but not unlimited.
228 # When you hit these limits, it's a signal to refactor - not a hard stop.
229 # =====
230
231 # =====
232 # Maximum number of arguments per function signature
233 # =====
```

Report

96 statements analyzed.

Statistics by type

| Type     | number | old number | difference | documented | loadname |
|----------|--------|------------|------------|------------|----------|
| module   | 11     | INC        | INC        | 100.00     | 0.00     |
| class    | 0      | INC        | INC        | 0          | 0        |
| method   | 0      | INC        | INC        | 0          | 0        |
| function | 0      | INC        | INC        | 100.00     | 0.00     |

External dependencies

```
..
requests (onboard)
```

194 lines have been analyzed

Raw metrics

| Type      | number | %     | previous | difference |
|-----------|--------|-------|----------|------------|
| code      | 128    | 65.22 | INC      | INC        |
| docstring | 23     | 12.58 | INC      | INC        |
| comment   | 8      | 4.35  | INC      | INC        |
| empty     | 33     | 17.83 | INC      | INC        |

Duplication

|                          | now   | previous | difference |
|--------------------------|-------|----------|------------|
| no duplicated lines      | 0     | INC      | INC        |
| percent duplicated lines | 0.000 | INC      | INC        |

Messages by category

| Type       | number | previous | difference |
|------------|--------|----------|------------|
| convention | 0      | INC      | INC        |
| refactor   | 0      | INC      | INC        |
| warning    | 1      | INC      | INC        |
| error      | 0      | INC      | INC        |

% errors / warnings by module

| module  | error | warning | refactor | convention |
|---------|-------|---------|----------|------------|
| onboard | 0.00  | 100.00  | 0.00     | 0.00       |

Messages

| message id                     | occurrences |
|--------------------------------|-------------|
| f-string-without-interpolation | 1           |

Your code has been rated at 9.90/10

What did the AI get right? What would you change? Did it include any rules that are too strict or too lenient for beginners?

What the AI got right:

- It correctly explained the purpose of a `.pylintrc` file and how it is used to configure Pylint.
- Showed the proper command to generate a starter configuration (`pylint --generate-rcfile > .pylintrc`).
- Highlighted useful customization options such as disabling warnings, setting maximum line length, and controlling naming conventions.
- Emphasized best practices like keeping `.pylintrc` under version control so teams share consistent rules.

### What I would change:

- The AI's explanation was thorough, but for an assignment answer it could be **more concise** and focused on the essentials (purpose, creation, usage).
- Instead of listing too many advanced options (like regex for variable names), a simpler example would be clearer for beginners.
- I would include a **sample beginner-friendly** `.pylintrc` **snippet** with only the most common rules (line length, disabling docstring warnings, etc.) to make it more practical.

### Rules too strict or too lenient for beginners:

- **Too strict:**
  - Enforcing complex naming regex rules may overwhelm beginners.
  - Requiring docstrings for every function/class is good practice but can be discouraging for those just learning syntax.

- **Too lenient:**

- Allowing very long lines (default 100+) might make code harder to read; beginners benefit from learning shorter line lengths early.
- Disabling too many warnings (like `invalid-name`) could prevent beginners from learning proper naming conventions.